

# 第十次作业反馈

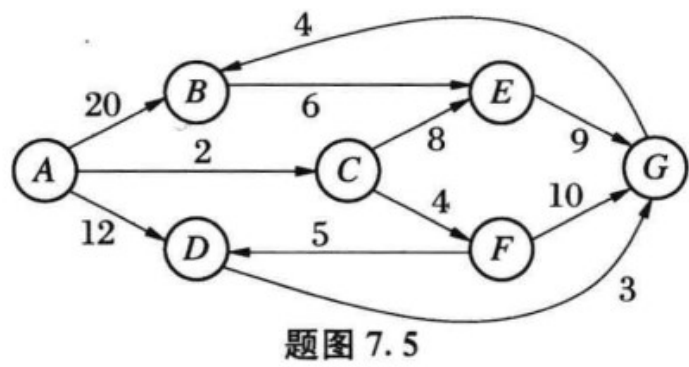
魏远毅

2024 年 11 月 28 日

## 1 问题 6.5

### 1.1 问题描述

试利用 Dijkstra 算法求解题图中顶点 A 到其他各顶点之间的最短路径。要求写出执行算法过程中数组 D、P 和 S 各步的状态。



### 1.2 解答

D 表示当前状态下源点到顶点 v 的最短路径，S 表示是否已经确定了源点到顶点 v 的最短路径，P 表示路径前驱顶点或最短路径。

|       |    |   |          |      |                 |    |   |          |         |
|-------|----|---|----------|------|-----------------|----|---|----------|---------|
| 初始状态: | 顶点 | S | D        | P    | → 选择最近顶点, 更新状态: | 顶点 | S | D        | P       |
|       | A  | 1 | 0        | -    |                 | A  | 1 | 0        | -       |
|       | B  | 0 | 20       | A->B |                 | B  | 0 | 20       | A->B    |
|       | C  | 0 | 2        | A->C |                 | C  | 1 | 2        | A->C    |
|       | D  | 0 | 12       | A->D |                 | D  | 0 | 12       | A->D    |
|       | E  | 0 | $\infty$ | -    |                 | E  | 0 | 10       | A->C->E |
|       | F  | 0 | $\infty$ | -    |                 | F  | 0 | 6        | A->C->F |
|       | G  | 0 | $\infty$ | -    |                 | G  | 0 | $\infty$ | -       |

|   |    |   |    |            |   |    |   |    |            |   |    |   |    |               |
|---|----|---|----|------------|---|----|---|----|------------|---|----|---|----|---------------|
| → | 顶点 | S | D  | P          | → | 顶点 | S | D  | P          | → | 顶点 | S | D  | P             |
|   | A  | 1 | 0  | -          |   | A  | 1 | 0  | -          |   | A  | 1 | 0  | -             |
|   | B  | 0 | 20 | A->B       |   | B  | 0 | 20 | A->B       |   | B  | 0 | 20 | A->B          |
|   | C  | 1 | 2  | A->C       |   | C  | 1 | 2  | A->C       |   | C  | 1 | 2  | A->C          |
|   | D  | 0 | 11 | A->C->F->D |   | D  | 0 | 11 | A->C->F->D |   | D  | 1 | 11 | A->C->F->D    |
|   | E  | 0 | 10 | A->C->E    |   | E  | 1 | 10 | A->C->E    |   | E  | 1 | 10 | A->C->E       |
|   | F  | 1 | 6  | A->C->F    |   | F  | 1 | 6  | A->C->F    |   | F  | 1 | 6  | A->C->F       |
|   | G  | 0 | 16 | A->C->F->G |   | G  | 0 | 16 | A->C->F->G |   | G  | 0 | 14 | A->C->F->D->G |

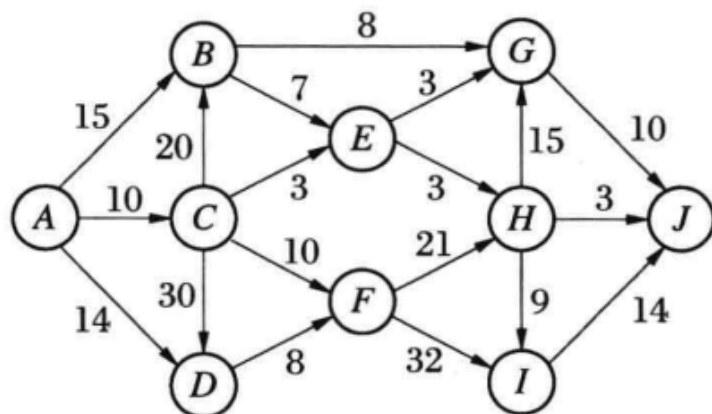
|   |    |   |    |                  |         |    |   |    |                  |
|---|----|---|----|------------------|---------|----|---|----|------------------|
| → | 顶点 | S | D  | P                | → 最终状态: | 顶点 | S | D  | P                |
|   | A  | 1 | 0  | -                |         | A  | 1 | 0  | -                |
|   | B  | 0 | 18 | A->C->F->D->G->B |         | B  | 1 | 18 | A->C->F->D->G->B |
|   | C  | 1 | 2  | A->C             |         | C  | 1 | 2  | A->C             |
|   | D  | 1 | 11 | A->C->F->D       |         | D  | 1 | 11 | A->C->F->D       |
|   | E  | 1 | 10 | A->C->E          |         | E  | 1 | 10 | A->C->E          |
|   | F  | 1 | 6  | A->C->F          |         | F  | 1 | 6  | A->C->F          |
|   | G  | 1 | 14 | A->C->F->D->G    |         | G  | 1 | 14 | A->C->F->D->G    |

## 2 问题 6.6

### 2.1 问题描述

一个有向无环网如题图所示，求：

- (1) 给出它的一个拓扑排序；
- (2) 计算每个顶点的最早发生时间和最迟发生时间；
- (3) 计算每条弧的最早开始时间和最迟开始时间；
- (4) 给出它的关键路径；缩短某些弧的持续时间将使关键路径变短，请给出这些弧。



题图 7.6

### 2.2 解答

1. 拓扑排序的基本方法：

- (1) 在图中任选一个入度为 0 的顶点，输出该顶点；
- (2) 从图中删除该顶点，并删除该顶点的所有出边；
- (3) 重复 (1) 和 (2) 直到图中不再有入度为 0 的顶点；

(4) 如果图中还有顶点，则说明图中存在环，则不存在拓扑排序，否则得到拓扑序列。

从 A 出发，按照拓扑排序的基本方法，可以得到拓扑序列：

A->C->B->D->E->F->H->G->I->J

(注意：拓扑排序不是唯一的，我批改的时候没注意这点，部分同学的答案有批改错误)

## 2. 每个顶点的最早发生时间和最迟发生时间的计算方法：

(1) 最早发生时间：

$$\begin{cases} ve[1] = 0 \\ ve[k] = \max\{ve[j] + d(v_j, v_k)\}, \quad v_j \rightarrow v_k \end{cases}$$

按照拓扑序列的顺序次序执行上述递推公式即可

(2) 最迟发生时间：

$$\begin{cases} vl[n] = ve[n] \\ vl[k] = \min\{vl[j] - d(v_k, v_j)\}, \quad v_k \rightarrow v_j \end{cases}$$

按照拓扑序列的逆序次序执行上述递推公式即可

从 A 开始：

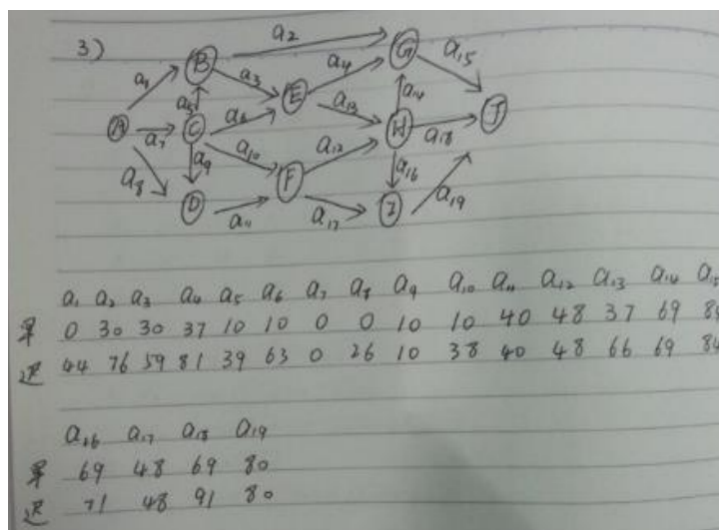
| 顶点     | A | B  | C  | D  | E  | F  | G  | H  | I  | J  |
|--------|---|----|----|----|----|----|----|----|----|----|
| 最早发生时间 | 0 | 30 | 10 | 40 | 37 | 48 | 84 | 69 | 80 | 94 |
| 最迟发生时间 | 0 | 59 | 10 | 40 | 66 | 48 | 84 | 69 | 80 | 94 |

## 3. 每条弧的最早开始时间和最迟开始时间的计算方法：

(1) 最早开始时间： $e(i, j) = ve[i], \quad v_i \rightarrow v_j$

(2) 最迟发生时间： $l(i, j) = vl[j] - d(v_i, v_j), \quad v_i \rightarrow v_j$

从 A 开始：



## 4. 关键路径的计算：

在完成上述 1、2、3 步骤后，找到所有满足  $e(i, j) = l(i, j)$  的边  $v_i \rightarrow v_j$ ，这些边构成了图的关键路径。

从 A 开始，关键路径为：A->C->D->F->H->G->J, A->C->D->F->I->J。

其中，只有缩短所有关键路径的公共路径才能使得关键路径变短（缩短非公共路径只会使得部分关键路径不再关键）

因此，我们需要缩短 A->C->D->F 这三条弧的持续时间。

### 3 问题 6.12

#### 3.1 问题描述

假设有向图以邻接表作为存储结构。试基于图的广度优先搜索策略实现一个算法，判断有向图中是否存在由顶点  $V_i$  到顶点  $V_j$  的简单路径。

#### 3.2 解答

这个问题并不困难，只需要了解广度优先搜索策略就可以解决。

下面给出基于队列的实现方式：

```
1  bool BFS(int s, int t){
2      queue<TreeNode*> q;
3      bool visited [MAXN] = {false};
4      visited [s] = true;
5      q.push(s);
6      while(!q.empty()){
7          TreeNode* u=AdjList[q.front ()];
8          q.pop();
9          for (TreeNode* p = u->firstarc; p!= NULL; p = p->nextarc){
10             v = p->adjvex;
11             if (v==t) return true;
12             if (! visited [v]){
13                 visited [v] = true;
14                 q.push(v);
15             }
16         }
17     }
18 }
```

### 4 问题 7.3

#### 4.1 问题描述

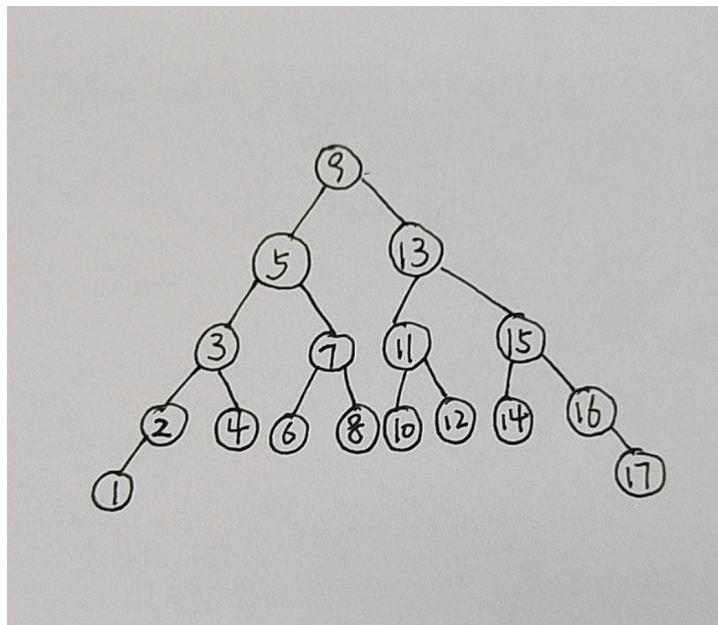
画出对长度为 17 的有序表进行折半查找的判定树，并分别求其等概率时查找成功的 ASL。

#### 4.2 解答

绘制二叉判定树如下：

$$ASL = (1+2*2+3*4+4*8+5*2)/17=59/17=3.47$$

$$(ASL=(\sum \text{每层深度} * \text{每层结点数})/\text{总结点数})$$



## 5 问题 7.12

### 5.1 问题描述

写出判别一颗二叉树是否为二叉排序树的算法，设二叉排序树中不存在关键字值相同的结点。

### 5.2 解答

通过中序遍历读取一棵二叉排序树，可以得到递增的序列，因此判断二叉树是否为排序树，只需要判断中序遍历结果是否递增。

下面给出两种实现算法 (本质相同，但后者更快):

```

1  elemType arr[MAXN]; // 存放中序遍历结果
2  int k=0; // 记录访问过的结点数
3
4  void Inorder_Traversal (TreeNode* T){
5      if (!T) return;
6      Inorder_Traversal (T->left);
7      arr[k] = T->val; // 记录结点值
8      k++;
9      Inorder_Traversal (T->right, arr);
10 }
11
12 bool Is_Binary_Sort_Tree(TreeNode* T){
13     Inorder_Traversal (T); // 先中序遍历
14     for (int i=1; i<k; i++)
15         if (arr[i] <= arr[i-1]) // 判断是否递增
16             return false;
17     return true;
18 }
  
```

```

1  elemType *pre = NULL; // 记录中序前驱节点的值
2
  
```

```
3  bool Is_Binary_Sort_Tree(TreeNode* T){
4      if (!T) return true;
5      if (!Is_Binary_Sort_Tree(T->left)) return false;
6
7      if (!pre) pre = &T->val; // 第一次访问节点
8      else if(*pre >= T->val) return false;
9      else pre = &T->val;
10
11     if (!Is_Binary_Sort_Tree(T->right)) return false;
12     return true;
13 }
```